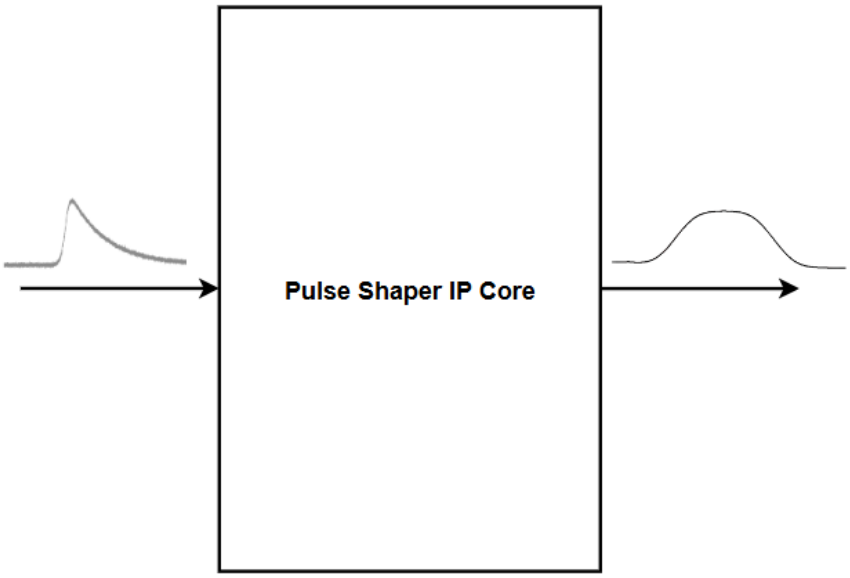


# Trapezoidal Pulse Shaper IP Core

## User Guide

---

IP Version: v2.4
August 28, 2026

**Author:**
Aldo Roberto Lupio-Guzman

# Contents

# List of Figures

# List of Tables

# Acronyms in This Document

| Acronym | Definition |
|---|---|
| ADC | Analog to Digital Converter |
| BRAM | Block Random Access Memory |
| CFD | Constant Fraction Discriminator |
| DSP | Digital Signal Processing block |
| FPGA | Field Programmable Gate Array |
| ILA | Integrated Logic Analyzer |
| IP | Intellectual Property core |
| LSB | Least Significant Bit |
| LUT | Look-Up Table |
| MSB | Most Significant Bit |
| RTL | Register Transfer Level |
| TCL | Tool Command Language |
| THS | Total Hold Slack |
| TNS | Total Negative Slack |
| VHDL | VHSIC Hardware Description Language |
| VIO | Virtual Input/Output core |

# 1. Introduction

The **Trapezoidal Pulse Shaper IP Core** processes the sample stream of an ADC connected to a radiation detector. For every pulse that arrives it **produces a trapezoidal shaped signal, measures the amplitude of that trapezoid and measures the rise time (10% to 90%)** of the original pulse. Amplitude and rise time together allow the user to identify the particle that produced the pulse.

The filter is a **streaming module**. It takes one sample per clock cycle and gives one result per clock cycle, without ever stopping. While it runs, it also follows the baseline of the signal and subtracts it, so a slow drift of the detector does not shift the measured amplitude.

The core **detects the pulses** on its own. A short Jordanov filter and a constant fraction discriminator find the instant at which each pulse arrives, in the same way for a small pulse and for a large one. Every event found this way is written into an internal BRAM together with a timestamp, where the user can extract the results later through a second memory port.

**Every pulse event is logged**, but accompanied by flags that say whether its amplitude and its rise time can be trusted. A pulse that arrives before the previous finishes it's decay is considered a pileup event. The core recognizes it, marks it as not usable and counts it.

The design is written in **plain VHDL** and uses no vendor primitives, so the same source can be used on Lattice, AMD and Red Pitaya boards. This guide describes the signals and the parameters of the core first, so that it can be connected and used quickly. The internal architecture, the equations and the validation are shown later, in chapters 5 and 6.

## 1.1 Features

- Trapezoidal shaping based on the recursive Jordanov algorithm.
- Automatic pulse detection with a fast filter and a constant fraction discriminator.
- Amplitude measurement averaged at the middle of the flat top.
- Rise time (10% to 90%) measurement averaged on the original pulse.
- Baseline tracking and averaged baseline subtraction.
- Pileup detection and pileup event counter.
- Event logging in an internal dual port memory with timestamp.
- Overflow flags for every arithmetic stage.
- One sample per clock, single clock domain.
- Device independent VHDL, no primitives.

## 1.2 Conventions

**Table 1.1.** Notation used in this document

| Notation | Meaning | Example |
|---|---|---|
| `NAME_I`, `NAME_O` | Input port, output port | `DATA_I`, `PULSE_STATE_O` |
| `_N` suffix | Active low | `RST_N_I` |
| `G_` prefix | Generic, fixed at synthesis | `G_ADC_WIDTH` |
| `C_` prefix | Constant of `trap_filter_pkg` | `C_LOG_DATA_WIDTH` |
| Sample | One sample equals one clock cycle | 128 samples is 1.02 µs at 125 MHz |
| Bit numbering | Bit 0 is the least significant bit | `PULSE_STATE_O`(0) |

# 2. Quick Facts

This chapter summarizes what the core is able to do. All the figures in brackets assume a sampling clock of 125 MHz. The repository and its files can be found in the following URL:

https://github.com/mexarlg/trap_filter

**Shaping**

- Converts the exponentially decaying pulse of a charge sensitive preamplifier into a trapezoid, using the recursive Jordanov algorithm.
- Rise time of the trapezoid adjustable from 16 to 256 samples (0.13 µs to 2.05 µs). A longer rise time averages more samples and gives a lower noise on the amplitude.
- Flat top adjustable from 16 to 256 samples (0.13 µs to 2.05 µs), to cover the charge collection time of the detector.
- Cancels the decay of the preamplifier for time constants up to about 4096 samples (32.8 µs), so that the trapezoid returns to zero and the amplitude does not depend on the tail of the previous pulse.
- Tracks the baseline continuously and subtracts it, so a slow drift of the detector does not shift the measured amplitude.

**Measurement**

- Measures the amplitude of every pulse on the flat top of the trapezoid, which is proportional to the energy deposited by the particle.
- Measures the rise time of every pulse on the original signal, which allows the host to tell one type of particle from another.
- Both results are averaged over a few samples to reduce the effect of the noise.
- Marks every event as clean or not clean, so that a doubtful measurement is never confused with a valid one.

**Rate and pileup**

- Processes one sample per clock cycle without interruption.
- Two pulses can be separated as long as they are further apart than the length of one trapezoid $(2k + m)$ plus its latency and decay time.
- Detects the pulses on its own, with a constant fraction discriminator, so the arrival time is found in the same way for small and for large pulses. No external trigger is needed.
- Detects and counts the pulses that arrive too close to each other, and flags them so that the user can reject them.

**Results and integration**

- Stores every event with a timestamp in an internal memory, 1024 events by default, which the user can read while the core keeps acquiring.
- Reports the saturation of every arithmetic stage after reset de-assertion.
- Written in plain VHDL with no vendor primitives, and validated on AMD Zedboard and Red Pitaya.

> **Note.** The Trapezoidal Pulse Shaper IP Core is aimed at processing pulses in a range from 50 ns to 250 ns of rise time. Other characteristics have not been tested and could require a change on the RTL.

> **Important.** The pulse detection subsystem performs under certain limitations: a very slow pulse can only be detected if it is large enough, and a very small pulse can only be detected if it is fast enough. Both characteristics are coupled. A fast and small pulse could produce a trigger that lies below the noise threshold, which would deny its detection and its logging. Currently, pulses higher than 600 LSBs are detected for very slow rising edges, under 100 LSBs of noise at 125 MHz. Under the expected pulse conditions mentioned previously on the Note, there are no detection issues.

# 3. Interface Description

This chapter describes everything needed to connect the core: the signals, the parameters, and the meaning of the encoded output vectors.

## 3.1 Top-Level Interface

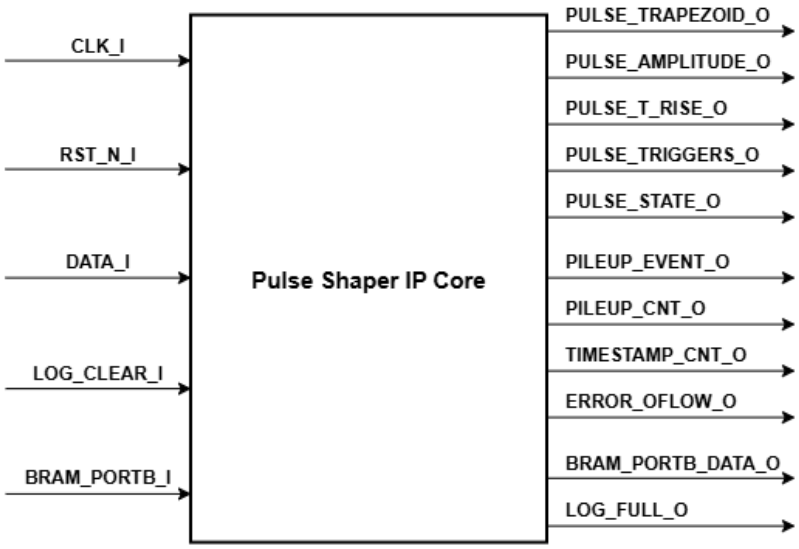


**Figure 3.1.** Top level interface diagram of the Trapezoidal Pulse Shaper IP Core

## 3.2 Signal Descriptions

In table 3.1 the width of a signal is written with a letter when it depends on a generic: $N$ is `G_ADC_WIDTH`, $A$ is `G_LOG_ADDR_WIDTH` and $P$ is `G_PILEUP_CNT_WIDTH`.

**Table 3.1.** Pulse Shaper IP Core Signal Descriptions

| Signal | Dir. | Width | Description |
|---|---|---|---|
| *Clock and reset* | | | |
| `CLK_I` | In | 1 | System clock. All logic is synchronous to its rising edge. |
| `RST_N_I` | In | 1 | Reset, active low. |
| *Input data* | | | |
| `DATA_I` | In | $N$ | Unsigned sample from the ADC. One new sample is expected on every clock cycle. |
| *Log memory, port B (User side)* | | | |
| `LOG_CLEAR_I` | In | 1 | Resets logger BRAM pointer. |
| `BRAM_B_EN_I` | In | 1 | Port B enable. |
| `BRAM_B_RW_I` | In | 1 | Port B read / write select. |
| `BRAM_B_ADDR_I` | In | $A$ | Port B address. |

| Signal | Dir. | Width | Description |
|---|---|---|---|
| BRAM_B_PULSE_DATA_O | Out | 32 | Data read from the pulse memory. Format in table 3.6. |
| BRAM_B_TIME_DATA_O | Out | 32 | Data read from the timestamp memory. |
| *Pulse outputs* | | | |
| PULSE_TRAPEZOID_O | Out | $N$+1 | Trapezoid produced by the filter, signed. Updated on every clock cycle. |
| PULSE_AMPLITUDE_O | Out | $N$+1 | Amplitude of the last measured pulse, signed. Held until the next event. |
| PULSE_T_RISE_O | Out | 12 | Rise time of the last measured pulse, unsigned, in samples. |
| PULSE_TRIGGERS_O | Out | 7 | Timing triggers of the current pulse. See table 3.3. |
| PULSE_STATE_O | Out | 3 | Validity of the last event. See table 3.4. |
| *System outputs* | | | |
| PILEUP_EVENT_O | Out | 1 | Trigger when a pileup is detected. |
| PILEUP_CNT_O | Out | $P$ | Number of pileup events since the reset was released. |
| TIMESTAMP_CNT_O | Out | 48 | Free running time counter, one step per clock cycle. |
| ERROR_OFLOW_O | Out | 10 | Overflow flags of the arithmetic stages. See table 3.5. |
| *Log memory, port A (monitoring)* | | | |
| LOG_FULL_O | Out | 1 | Full log BRAM flag. |
| LOG_PULSE_DATA_O | Out | 32 | Word written into the pulse memory for the current event. |
| LOG_TIMESTAMP_DATA_O | Out | 32 | Word written into the timestamp memory for the current event. |

**Note.** The core has no data valid input. Every value present on DATA_I is treated as a valid sample. If the ADC produces samples at a lower rate than the clock, the input must be resampled before the core.

## 3.3 Parameters

All the parameters of the core are VHDL generics.

**Table 3.2.** Pulse Shaper IP Core Generic Descriptions

| Generic | Range | Default | Description |
|---|---|---|---|
| *Input data* | | | |
| G_ADC_WIDTH | 12–15 | 14 | Width of the ADC sample. |
| *Trapezoidal filter* | | | |
| G_SLOW_JORD_K_DELAY | 16–256 | 128 | Rise time $k$ of the trapezoid, in samples. |
| G_SLOW_JORD_M_DELAY | 16–256 | 256 | Flat top $m$ of the trapezoid, in samples. |

| Generic | Range | Default | Description |
|---|---|---|---|
| G_SLOW_JORD_M_EXP_VALUE | 0–65535 | 39992 | Decay correction term $M$, in fixed point with 12 integer bits and 4 fractional bits. |
| *Moving averages* | | | |
| G_BASE_MOV_DELAY_WIDTH | 3–5 | 4 | Baseline average length as a power of two. A value of 4 means 16 samples. |
| G_PEAK_MOV_DELAY_WIDTH | 3–5 | 3 | Amplitude average length as a power of two. |
| G_T_RISE_MOV_DELAY_WIDTH | 3–5 | 3 | Rise time average length as a power of two. |
| *Detection* | | | |
| G_NOISE_THRESHOLD | 10–4096 | 100 | Level the signal must cross before a pulse is accepted, in ADC lsb. |
| G_PILEUP_DECAY_VALUE | 255–65535 | 2500 | Samples of decay time during which a new pulse is treated as a pileup. |
| *Logger* | | | |
| G_LOG_ADDR_WIDTH | 10–16 | 10 | Address width of the log memory. It holds $2^N$ events. |
| G_PILEUP_CNT_WIDTH | 12–24 | 24 | Width of the pileup counter. |
| G_TIMESTAMP_DIV | 0–6 | 4 | Bits removed from the timestamp. A larger value gives a longer range and a coarser step. With 4, one step is 128 ns at 125 MHz. |

> **Important.** All the parameters are fixed when the design is synthesized. To change the shaping parameters the design must be synthesized again.

## 3.4 Encoded Output Signals

Four signals carry several fields inside one vector. The tables below give the meaning of each bit.

**Table 3.3.** Encoding of PULSE_TRIGGERS_O

| Bit | Name | Meaning |
|---|---|---|
| 6 | PULSE_DETECTED | A pulse has been detected. |
| 5 | BASELINE | The baseline is subtracted at this instant. |
| 4 | PULSE_START | Start of the rising edge of the trapezoid. |
| 3 | TOP_START | Start of the flat top of the trapezoid. |
| 2 | TOP_MID | Middle of the flat top. The amplitude is captured here. |
| 1 | PULSE_END | End of trapezoid and timeout for rise time capture. |
| 0 | LOG_EVENT | The event is written into the log memory. |

**Table 3.4.** Encoding of `PULSE_STATE_O`

| Bit | Name | Meaning |
|---|---|---|
| 2 | `ALL_VALID` | Both results are valid and no overflow occurred. |
| 1 | `AMPLITUDE_VALID` | The amplitude can be used. |
| 0 | `T_RISE_VALID` | The rise time can be used. |

> **Important.** A valid variable means that its capture process was not interrupted, that it was not affected by overflow errors or pileup, and that it was not measured with incomplete delay lines. It does not mean that the values are physically meaningful, since the module parameters could be erroneously set for the input pulses.

**Table 3.5.** Encoding of `ERROR_OFLOW_O`

| Bits | Source | Meaning |
|---|---|---|
| 9–8 | Trapezoid subsystem | Overflow in the slow Jordanov accumulators 1 and 2. |
| 7 | Trapezoid subsystem | Overflow in the baseline moving average. |
| 6 | Trapezoid subsystem | Overflow in the baseline subtraction. |
| 5–4 | Trigger subsystem | Overflow in the fast Jordanov accumulators 1 and 2. |
| 3–2 | Trigger subsystem | Overflow in the constant fraction discriminator. |
| 1 | Capture subsystem | Overflow in the amplitude moving average. |
| 0 | Capture subsystem | Overflow in the rise time moving average. |

**Table 3.6.** Encoding of the BRAM pulse log

| Bits | Field | Description |
|---|---|---|
| 31–16 | `AMPLITUDE` | Amplitude of the pulse, signed, 16 bits maximum. |
| 15 | `AMP_VALID` | The amplitude of this event can be used. |
| 14–13 | – | Not used. |
| 12–1 | `T_RISE` | Rise time of the pulse, unsigned, in samples. |
| 0 | `T_RISE_VALID` | The rise time of this event can be used. |

The timestamp word holds the value of the internal 48 bit counter, shifted right by `G_TIMESTAMP_DIV` bits and truncated to 32 bits. The `ALL_VALID` bit is not stored so the user should obtain it as the logical AND of bit 15 and bit 0 of the pulse data log.

> **Important.** The bit positions in table 3.6 are allocated for the largest data width the core supports, not for the width actually configured. The amplitude field always occupies bits 31 to 16, even though the amplitude is only `G_ADC_WIDTH`+1 bits wide, so with the default settings 15 of those 16 bits carry information (LSBs are padded). The same logic is applied to the rise time. Check the package for more details.

> **Note.** The timestamp of a logged event is captured at the moment of logging, that is when `LOG_EVENT` is asserted. The user should subtract this value with the time that passed from the pulse detection to the log event to correctly describe it detection timestamp. In addition, another option would be to also subtract the latency from the pulse detection module to have a timestamp relative to the input data. Take into consideration the value of `G_TIMESTAMP_DIV` for this operation.

# 4. Using the IP Core

## 4.1 Adding the IP to a Project

The repository contains TCL scripts in the `scripts` folder that build everything automatically, so no file has to be added by hand. Depending on the purpose, there are 5 possible options:

- **Testing the design on the ZedBoard.** To see the filter running on a board without any extra setup, select the window **Tools** → **Run TCL Script** or run the following:

```
vivado -mode batch -source <path>/trap_filter/scripts/build_zedboard_trap_filter.tcl
```

  This creates a complete project with the RTL, the package, the virtual pulse and the test wrapper compiled into the library `trap_filter`, ready to simulate and to program.

- **Adding the IP from any Vivado open project.** If a generic project is already open, select the following script in **Tools** → **Run TCL Script** or from the TCL console:

```
source <path>/trap_filter/scripts/generate_pulse_shaper_ip.tcl
```

  The script packages the core, adds the repository to the open project and refreshes the catalog.

- **Creating the IP without a Vivado project open.** To obtain the core as a block that can be dropped into any design, select the following script in **Tools** → **Run TCL Script** or run:

```
vivado -mode batch -source <path>/trap_filter/scripts/generate_pulse_shaper_ip.tcl
```

  This generates `ip/trap_filter/component.xml`. To make it visible later in a project, open that project and enter the two following commands in the TCL console:

```
set_property ip_repo_paths <path>/trap_filter/ip [current_project]
update_ip_catalog -rebuild
```

  The core then appears under **IP Catalog** → **User Repository** → **UserIP** or in the search bar inside block design, as seen in fig. 4.1.

- **Adding the IP to the Red Pitaya ADC project.** To generate the project to test the Trapezoidal Pulse Shaper IP Core with a real live implementation using the ADC from Red Pitaya, select the following script in **Tools** → **Run TCL Script** or run the following command:

```
vivado -mode batch -source <path>/trap_filter/scripts/build_redpitaya_trap_filter.tcl
```

  It will generate the block design from the IP sources and add a top. If the board is not installed, add the /boards folder as a board repository in the Vivado project settings. If an error version appears, delete the Vivado version block code in the script.

- **Adding the IP to a lattice CertusPro-NX100 Radiant project.** Radiant software does not allow IP packaging in the same way as Vivado. In order to generate the top file of the Trapezoidal Pulse Shaper IP Core without any additional module, run the following command:

```
source <path>/trap_filter/scripts/build_lattice_trap_filter.tcl
```

  Take into consideration that Radiant implementation reports will not be available unless the number of I/O ports of the IP is reduced. To do this, either create a dummy wrapper.

> **Note.** The structure of the folders must stay the same, as the IP core refers to the sources in place and does not copy them. The IP has to be generated again whenever a port or a generic of `pulse_shaper_top` changes. If only the internal architecture is modified, run: `update_ip_catalog -rebuild`.

> **Important.** The `build_redpitaya_trap_filter.tcl` script relies on the manual connection of the output ports of the Trapezoidal Pulse Shaper IP Core to the rest of the block modules. If any of the ports are modified or changed its naming, the modifications should be applied to the script.

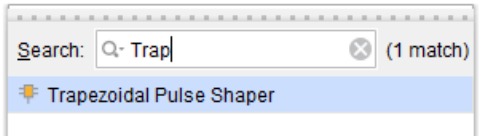


**Figure 4.1.** Trapezoidal Pulse Shaper IP Core IP found in the Vivado block design window.

Once the IP Core has been generated and added to the project, it should be found in the block design catalogue. A view of the final IP Core is shown in fig. 4.2.

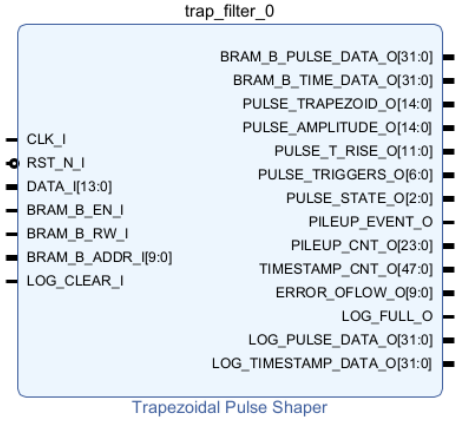


**Figure 4.2.** Top level interface of Trapezoidal Pulse Shaper IP Corefound in the Vivado block design window.

## 4.2 Configuring the IP Core

Every significant parameter selected in the generics corresponds to a physical characteristic extracted from the sampled pulse signals. For instance, Figure 4.3 shows the raw pulse at the input of the core, while Figure 4.4 shows the trapezoidal filtered data that outputs the IP Core. As seen, each of the Jordanov delays highlight the shape of the trapezoid. In order to set the core, the user should measure or know at least 5 variables from the real detector signal.

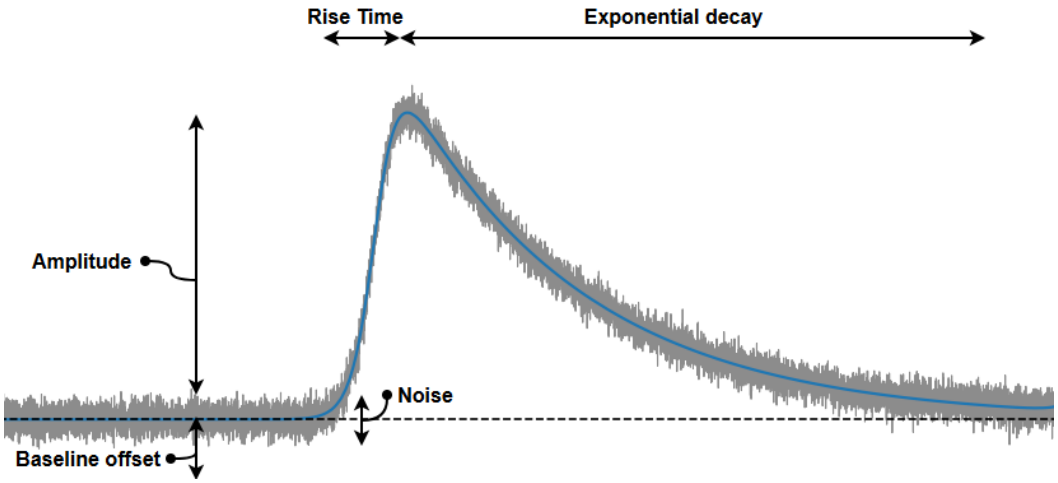


**Figure 4.3.** Raw pulse characteristics.

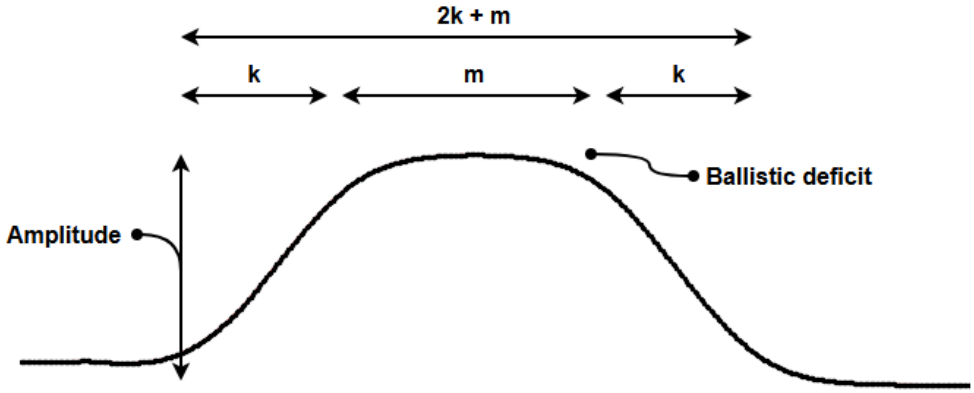


**Figure 4.4.** Filtered trapezoidal pulse characteristics.

**The summary** of the generic configuration inside the block design window can be seen in fig. 4.5:

- **The decay of the tail sets `G_SLOW_JORD_M_EXP_VALUE`.** This is the most important parameter. Measure the time constant $\tau$ of the exponential tail on the raw pulse, then convert it with eq. (4.1). The factor 16 is there because the generic carries four fractional bits:

$$\texttt{G\_SLOW\_JORD\_M\_EXP\_VALUE} = 16 \cdot M = \frac{16}{e^{T/\tau} - 1} \approx \frac{16\,\tau}{T} \quad \text{for } \tau \gg T \tag{4.1}$$

  The default value of 39992 corresponds to a decay of about 2500 samples, that is 20 µs at 125 MHz. If the value is too small, the trapezoid keeps a positive tail after the flat top. If it is too large, the trapezoid lower corner dips below zero. An example of this ballistic deficit is seen in fig. 4.4.

- **The noise deviation sets `G_NOISE_THRESHOLD`.** Record the baseline with no source in front of the detector and measure how far the noise swings in the ADC resolution. One can either decide to set the parameter as the baseline plus the noise, or just the noise. This parameter **gates** the possibility of having a zero crossing event on the CFD module from **pulses of amplitude smaller than this threshold**, so if it is set too low the noise will create false events, and if it is set too high the smaller pulses could be lost.

- **The charge collection time sets `G_SLOW_JORD_M_DELAY`.** The flat top must last longer than the time the detector needs to collect all the charge, which is the duration of the rising edge on the raw pulse. If the flat top is shorter, the top is not flat and the measured amplitude and rise time could be wrongly measured, especially if the averaged samples at the top are still located in the rising edge.

- **The rate and the pulse rise time set `G_SLOW_JORD_K_DELAY`.** The ramp $k$ is where the noise averaging happens, so a longer $k$ gives a cleaner amplitude. The price is a higher width of the trapezoid ($2k+m$). With the default values this is 512 samples, that is 4.1 µs at 125 MHz. One can choose $k$ so that $2k+m$ plus the decay time stay comfortably below the average distance between pulses, while making sure that $k+m$ is larger than the minimum rise time.

- **The tail decay length sets `G_PILEUP_DECAY_VALUE`.** This is the window, in samples, during which a new pulse is treated as a pileup. It is normally of the order of $\tau$. A longer window rejects more good events but makes the ones that survive cleaner. Remember that even if events are identified as pileup, they will be logged.

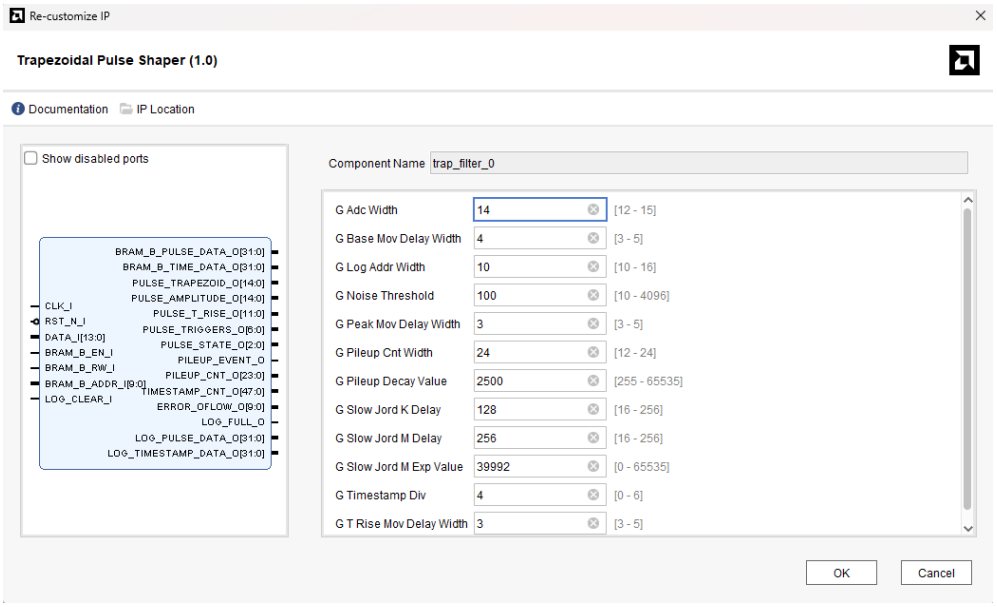


**Figure 4.5.** Generic default configuration in the Trapezoidal Pulse Shaper IP Core.

> **Note.** The remaining generics can keep their default value. The three moving average lengths only trade a little noise against more conservative values, while the logger parameters only depend on how often the user reads the memory.

## 4.3 Operation

**Reset and start-up.**

- **Release the reset only when the ADC can deliver samples.** The core starts to process whatever is present on `DATA_I` just after some cycles of latency, once `RST_N_I` is de-asserted. If the input is still held at a constant value at that moment, the step that appears when the real data finally arrives looks like a pulse and it could be measured as valid. This is true for a noise threshold without the baseline effect, which could be problematic. Since the pileup timeout would flag real pulses as pileup for `G_PILEUP_DECAY_VALUE` samples after the baseline issued pulse.

- **The outputs stream from the very first cycle, valid or not.** `PULSE_TRAPEZOID_O` is updated on every clock cycle, and the captured values are driven as soon as the logic produces them. Nothing

on the interface is held back during the start-up, so the outputs must not be trusted on their own. The state bits are what say whether a result can be used.

- **An event is marked as valid only when four conditions are met.**
  - The delay lines are full.
  - One full trapezoid plus latency have passed since the data started arriving.
  - No pileup was detected on that event.
  - No overflow flag were raised.
- **Once started, the core can be left running.** The filter is recursive but it does not drift numerically due to the own architecture of the Jordanov algorithm. However, margin bits and overflow flags could be asserted if the accumulators started to drift. Currently, the IP Core has been tested by a continuously run of more than 2 hours without any degradation of the output data or the logged values.

> **Note.** Resetting the core clears the pileup counter, the timestamp counter, the overflow flags, and it empties the delay lines and accumulators. A reset in the middle of an acquisition therefore costs the events still in the pipeline. However, a reset should still be asserted after changing any of the pulse characteristics in testing environments.

## 4.4 Troubleshooting

A set of common problems and their likely solutions can be found in table 4.1:

**Table 4.1.** Common problems found with real ADC data streaming

| Symptom | Likely cause | Action |
|---|---|---|
| No event is ever detected (1) | Threshold too high, or signed input | Lower `G_NOISE_THRESHOLD` and make sure `DATA_I` is unsigned. |
| No event is ever detected (2) | Detection system triggers falling below the noise threshold | Verify input pulses are not too small and slow for the current detection parameters (16 samples of rise). |
| Many events with no pulses present | Threshold too low | Raise `G_NOISE_THRESHOLD` above noise. |
| The pulse output data is inaccurate | Jordanov parameters are incorrect | Increase `G_SLOW_JORD_K_DELAY` or `G_SLOW_JORD_M_DELAY` and verify `G_SLOW_JORD_M_EXP`. |
| The first pulses are ignored | Decay timeout activated due to baseline pulse | Increase `G_NOISE_THRESHOLD` above baseline or delay incoming pulses. |
| A bit of `ERROR_OFLOW_O` stays high | An accumulator overflow | Use table 3.5 to find the stage and make sure input data is not signed. |
| `LOG_FULL_O` is high | Logger BRAM is full and no more events can be stored | Assert high `LOG_CLEAR_I` to restart BRAM pointer. |
| No outputs are being produced | Reset polarity | Verify the input reset signal is active-low. |
| Lagged or artificial looking data | Clock synchronization | Verify the input clock is the same clock being used by the ADC. |

# 5. Functional Description

This chapter explains how the core architecture has been developed. The core is divided into five subsystems, listed in table 5.1. The raw input goes to three of them in parallel: the trigger subsystem decides **when** a pulse happened, the trapezoid subsystem computes the **streaming of the filtered data**, and the capture subsystem **measures the results** using the triggers of the first one. The validity logic and the logging of the pulse events are left in the other subsystems.

## 5.1 Architecture

The detailed block diagram of the Trapezoidal Pulse Shaper IP Core can be seen in fig. 5.1. No external or test logic is shown apart from the IP. Some signal connections may not be shown for visual purposes.

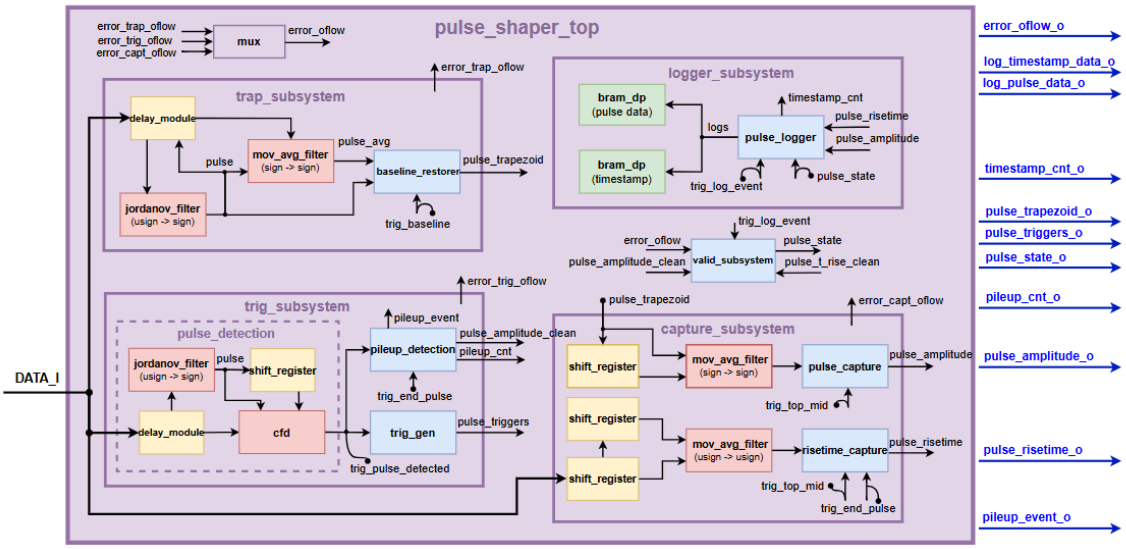


**Figure 5.1.** Functional block diagram of the Trapezoidal Pulse Shaper IP Core

A summary of the function of each subsystem is given below in table 5.1:

**Table 5.1.** Subsystems of the Trapezoidal Pulse Shaper IP Core

| Subsystem | Function |
|---|---|
| `trig_subsystem` | Detects the pulses with a short Jordanov filter and a CFD module to issue the seven triggers of table 3.3 that synchronize the whole system. Also detects and counts the pileup events. |
| `trap_subsystem` | Produces the streaming filtered trapezoid data. Contains the slow Jordanov filter, the moving average that follows the baseline, and the output data without the baseline. |
| `capture_subsystem` | Measures the amplitude on the flat top and the rise time on the raw input, and reports whether each one is clean. |
| `valid_subsystem` | Combines the clean flags, the delays and the overflow flags into `PULSE_STATE_O`. |
| `logger_subsystem` | Writes the amplitude, the rise time, the state and a timestamp into the dual port memory on the `LOG_EVENT` trigger. |

## 5.2 Jordanov Algorithm

The filter follows the recursive form given by Jordanov and Knoll [1]. Let $v(n)$ be the input sample at index $n$, $k$ the rise time in samples, $m$ the flat top in samples and $l = k + m$. The first step builds a difference of four delayed copies of the input:

$$d^{k,l}(n) = v(n) - v(n-k) - v(n-l) + v(n-k-l) \tag{5.1}$$

The second step accumulates that difference and adds a correction term that cancels the exponential decay of the preamplifier:

$$p(n) = p(n-1) + d^{k,l}(n) \tag{5.2}$$

$$s(n) = s(n-1) + p(n) + M\, d^{k,l}(n) \tag{5.3}$$

$s(n)$ is the trapezoid, and $M$ is the decay correction of eq. (4.1). Only additions and one multiplication are needed, and the cost does not grow when $k$ or $m$ grow. This is why the recursive form is used instead of the direct convolution.

The algorithm arithmetic in FPGA logic is done in 8 stages, one stage per clock cycle, so that a new sample can be accepted on every edge and the throughput is independent of the shaping parameters. The first five stages implement the filtering. The remaining four stages normalize the result, bringing it back from the internal precision of table 5.3 to the output width. Table 5.2 summaries the stages and the latency at which each result becomes valid.

**Table 5.2.** Pipeline stages of the Jordanov recursion

| Stage | Expression | Cycle | Implementation |
|---|---|---|---|
| *Filtering* | | | |
| $d[n]$ | $v[n] - v_k[n] - v_l[n] + v_{kl}[n]$ | 1 | Delayed difference |
| $a_1[n]$ | $a_1[n-1] + d[n]$ | 2 | First accumulator |
| $p[n]$ | $M \cdot d[n]$ | 2 | DSP multiplication |
| $a_1'[n]$ | $a_1[n] \ll f$ | 3 | Width and cycle alignment |
| $a_2[n]$ | $a_2[n-1] + a_1'[n] + p[n]$ | 4 | Second accumulator |
| *Normalization* | | | |
| $a_2'[n]$ | $a_2[n] \ll s$ | 5 | Shift to the DSP operand width |
| $q[n]$ | $a_2'[n] \cdot c$ | 6 | Division by the filter gain |
| $r[n]$ | $\mathrm{round}(q[n])$ | 7 | Rounding to the output width |
| $y[n]$ | $\mathrm{saturate}(r[n])$ | 8 | Saturation to $\pm$ full scale |

## 5.3 Pulse Detection and Triggers

The Trapezoidal Pulse Shaper IP Core finds the pulses by itself. It uses a second fast Jordanov filter, which is much faster than the one used for the filtering. Its rise time and its flat top are both 16 samples, so the detection of a pulse relies on this output being able to rise above the noise threshold, which might be an issue in very small and slow pulses. However, these kind of pulses are not expected, although the fast Jordanov delays could be increased in the package if the trigger delay constants are adapted accordingly,

The output of the fast Jordanov filter goes into a constant fraction discriminator. The CFD subtracts a scaled copy of the signal from a delayed copy of itself, and looks for the instant where the result crosses zero. That instant does not depend on the height of the pulse, so small and large pulses are timed in the same way.

The delay used inside the CFD is 32 samples, a slope check over 8 samples makes sure that only rising edges are accepted, and a timeout of 128 samples cancels the event if the expected zero crossing never

arrives (timeout needed due to an existing delay between the input data and the own CFD delays). A pulse is accepted only if the fast Jordanov output also crosses `G_NOISE_THRESHOLD`, which stops the noise of the baseline from creating false events.

From that instant, each trigger of table 3.3 is issued after a fixed delay. The baseline is captured 4 samples after the detection, and the rising edge of the slow trapezoid starts 30 samples after it (with the current 16 samples of fast Jordanov). Since the pulse detection and its triggering should be known beforehand, the same input data is delayed accordingly in the trap_subsystem.

The waveform in fig. 5.2 shows two consecutive events, the first pulse being valid and the second pulse affected by pileup. The raw input data is shown in white at the top, followed in cyan by the fast Jordanov output and the constant fraction discriminator signals. The stage triggers are shown in yellow, the pileup detection in pink, and the shaped output in salmon at the bottom.

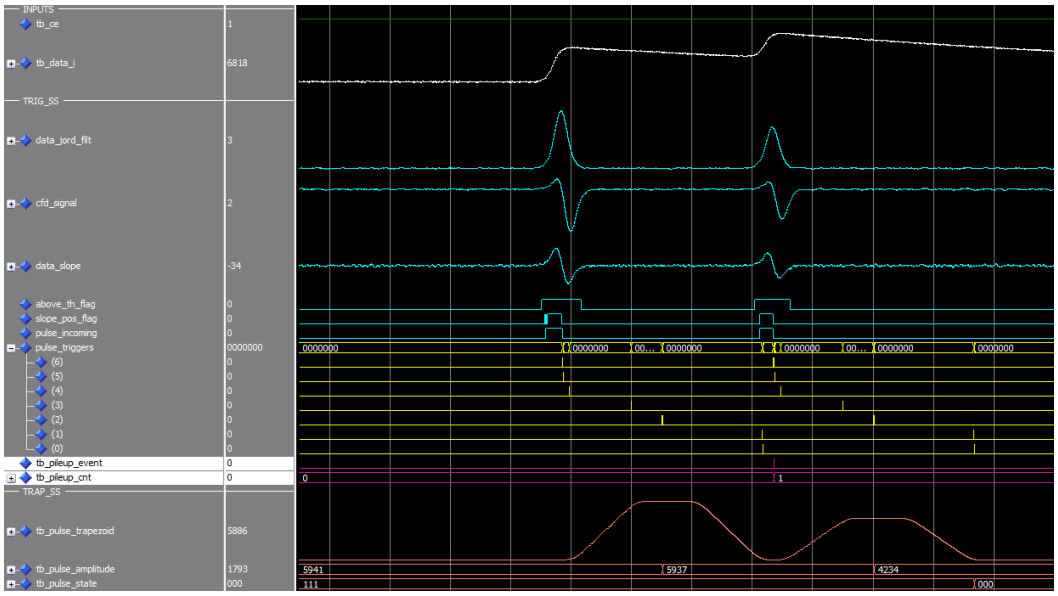


**Figure 5.2.** Timeline of trigger_subsystem from ModelSim waveform.

## 5.4 Measurement and Pileup

The amplitude is taken from the trapezoid at the middle of the flat top, on the `TOP_MID` trigger, and averaged over an amount of samples specified in the generics. The rise time is measured on the raw input, which is delayed inside the capture subsystem so that it lines up with the `TOP_MID` trigger. This occurs because the expected amplitude should be known before the rise time capture, in order to compute both 10% and 90% amplitude thresholds. A counter will be active between these ranges, but if the trapezoidal data is finished and the logging trigger is issued without the rise time being completed, the captured value will be logged as not clean. This protects against stalling or recording unfinished data as valid.

Regarding pileup detection, two pulses pile up when the second one arrives before the first one has finished its processing, its capture or its decay. Due to this effect the measured amplitude is wrong, since the second pulse sits on the tail of the first one. The core is able to detect this type of event, flag it as not clean and even increase the pileup counter and restart the new decay timeout. This is done through a finite state machine, which is able to log if a pulse is asserted on the middle of an ongoing decay, by keeping a short history of the previous pulse.

As shown in fig. 5.3, part of the pileup detection module is seen in action. At the top of the waveform lies the input pulse data. In light green, one can see the output of the fast Jordanov, the CFD and the slope of the input data. Right below in cyan blue, the logic signals describing the conditions and flags of the possible conditions for a pulse event detection. After them, in orange, the generated triggers for synchronization. And finally in yellow, the logic regarding pileup discrimination.

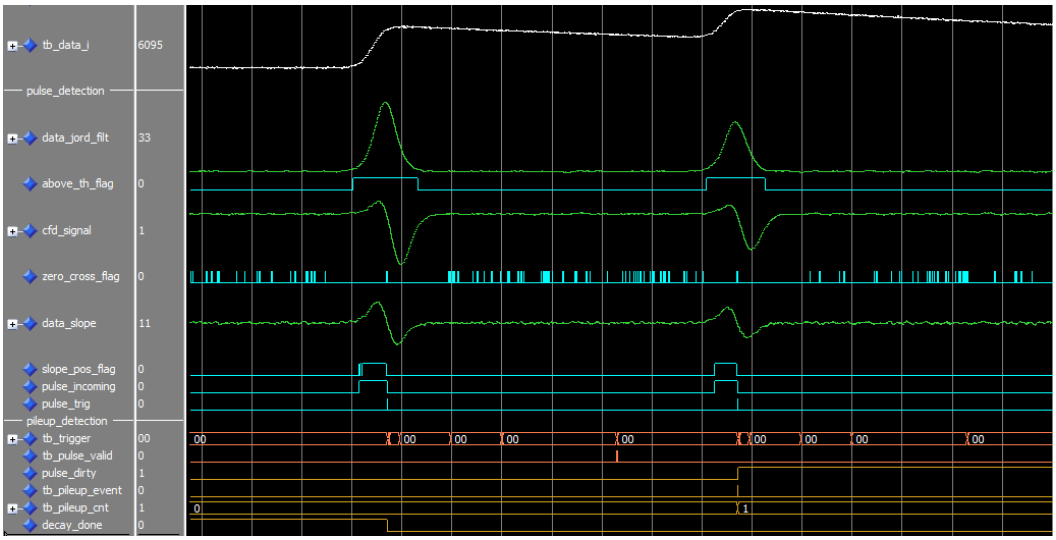


**Figure 5.3.** Timeline of pulse_detection and pileup_detection from ModelSim waveform.

## 5.5 Arithmetic and Overflow

The trapezoid and the amplitude are signed and one bit wider than the input, because the baseline is subtracted from the filtered data and the result can be negative. Inside the filters, each arithmetic stage is given a few extra margin bits depending on the criticality of the stage, so that the intermediate values do not saturate during normal operation. Those margins are constants of the package, listed in table A.1. If an arithmetic stage starts to use any of the margin bits, the system will detect it and flag it as overflow. The output data will be correct, but the overflow flag assertion should be treated as a warning that some of the stages might overflow soon, by actually corrupting the data outputs.

For instance, the trapezoidal recursion of Jordanov accumulates twice, so the internal signals grow considerably wider than the incoming samples. Each stage is sized from the width of the data it carries plus the growth introduced by the operation, and a small configurable margin on top, so that no intermediate result can wrap even for a sustained input at full scale. Table 5.3 lists the resulting widths for a 14 bit input and the default delays. Only the final output is truncated back to the input width plus a sign bit, the internal precision being discarded once the trapezoid has been formed.

**Table 5.3.** Widths of the arithmetic stages of the Jordanov for the default generics of `G_DATA_WIDTH` = 14 and `G_K_DELAY` = 128

| Signal | Width | Composition |
|---|---|---|
| *Pipeline* | | |
| `C_DIFF_WIDTH` | 18 | data (14) + sign (1) + margin (3) |
| `C_ACC1_WIDTH` | 25 | data (14) + sign (1) + $k$ (8) + margin (2) |
| `C_MDIFF_WIDTH` | 35 | $M_{exp}$ (17) × diff (18) |
| `C_ACC1_SC_WIDTH` | 29 | acc1 (25) + $M_{exp}$ fraction (4) |
| `C_ACC2_WIDTH` | 44 | $M\cdot$ diff (35) + margin (1) + $k$ (8) |
| *Output* | | |
| `C_DATA_FILTERED_WIDTH` | 15 | data (14) + sign (1) |

> **Note.** When a value does not fit, it is clipped to the largest or the smallest number that the stage can hold, and the corresponding bit of `ERROR_OFLOW_O` is raised if any of the margin bits are used. The event is still logged, but its state bits are not valid.

## 5.6 System Timeline

In fig. 5.4, it is shown a simplified version of the IP timeline. In dark blue, the input and delayed data. In green, the pulse detection and pileup signals, while red is used for the stage triggers. In light blue, the filtered stages of the trapezoid subsystem, and finally, the amplitude and rise time capture are represented in yellow and purple respectively.

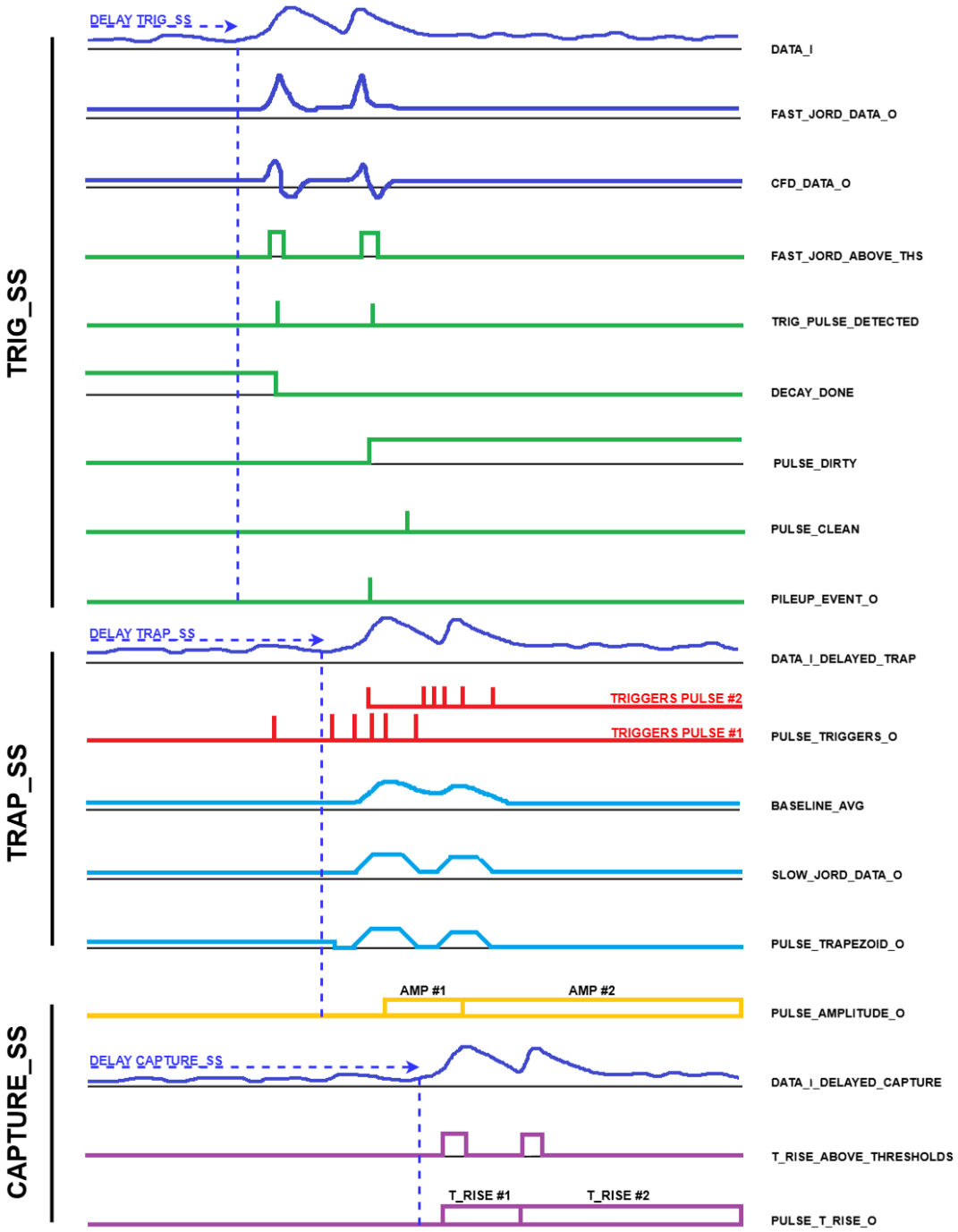


**Figure 5.4.** Timeline simplified summary where first pulse is valid while second event is affected by pileup.

> **Note.** Some of the delays are not very precise but rather give an overall idea of the stages of the system.

# 6. Simulation and Validation

The core was validated in three steps. First the RTL alone in simulation by introducing a python generated stimulus, then the RTL running on a device with the same stimulus stored in a ROM, and finally the RTL fed by real ADC data stream.

The virtual pulse stimulus is generated with the `pulse_gen` script in the `sw` folder. The user selects the physical characteristics of the pulse, such as amplitude, rise time, decay, noise, baseline offset and the pileup events to insert, and the script writes one window of samples as a plain `.txt` file and as a VHDL package. The package is compiled into the design and stored by the `pulse_feed` module, which replays the window sample by sample, simulating an ADC. The same package is therefore used in simulation and on the board, which is what makes the two directly comparable. Figures 6.1 and 6.2 show two of the sample windows used.

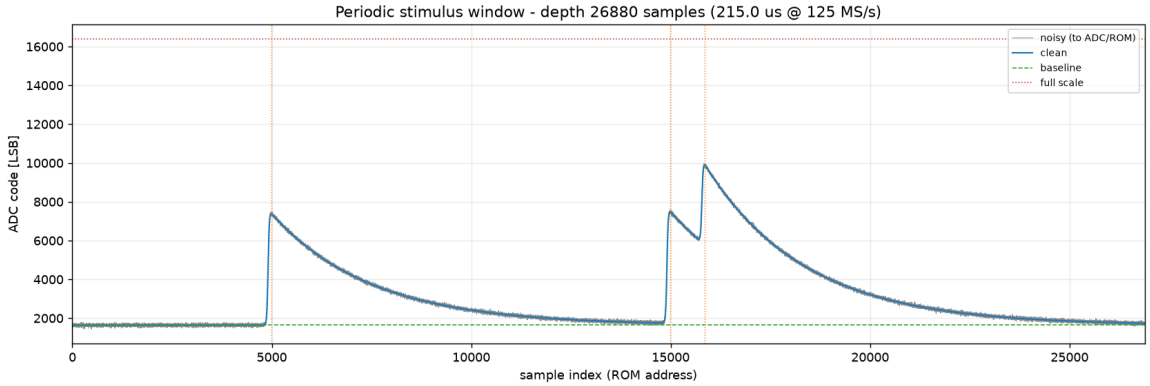


**Figure 6.1.** Input stimulus window with slow rising pulses, and added random noise and baseline offset. Amplitude at 35 % of maximum ADC width, rise time at 250 ns

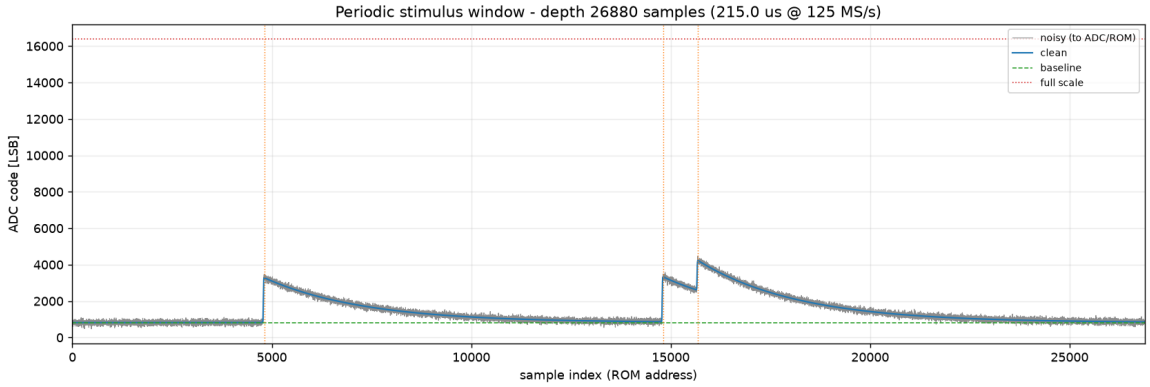


**Figure 6.2.** Input stimulus window with fast rising pulses, and added random noise and baseline offset. Amplitude at 15 % of maximum ADC width, rise time at 60 ns

## 6.1 Simulation in ModelSim

The first step runs the design in ModelSim with the virtual pulse as input. The unit under test is `pulse_shaper_test_wrapper` which was intended for the Zedboard. `pulse_feed` holds the pulse package and drives `DATA_I`, and the wrapper provides the clock and the reset sequence.

To run the simulation, open ModelSim and set its working directory to the `modelsim` folder inside `sim`, either with **File** → **Change Directory** or from the transcript:

```
cd <path>/trap_filter/sim/modelsim
```

Then compile the sources, elaborate the testbench and run it to generate the waveform (files with wave_ suffix naming):

```
do compile_pulse_shaper_top.do
do simulate_pulse_shaper_top.do
run -all
```

The first script compiles the package and all the RTL into the library `trap_filter`, the second loads the testbench and opens the waveform window with the signals already added, and `run -all` plays the whole stimulus window until the testbench `tb_pulse_shaper_top` stops the simulation.

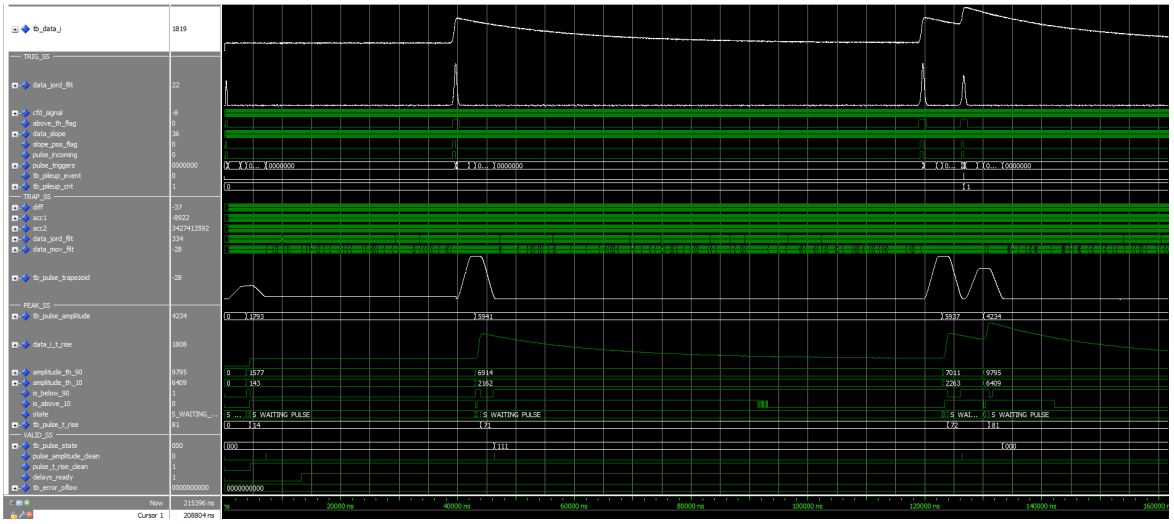

**Figure 6.3.** ModelSim waveform for `tb_pulse_shaper_top.vhd`.

## 6.2 ROM Stimulus Replay on the ZedBoard

The same wrapper is then programmed onto the Zedboard through the script mentioned in section 4.1, where `pulse_feed` replays the window in a loop. An ILA then captures the internal signals while the core runs, driven by VIO inputs. Since the stimulus is the one already used in ModelSim, the captured waveforms can be laid next to the simulated ones and compared point by point.

Looping the window also tests two things that a single simulation cannot verify. The filter is recursive, so an error in the accumulators or in the baseline restoration would drift and become visible after thousands of pulses. The second is repeatability, since the same pulse should give the same amplitude and the same rise time even after several loops. This test wrapper configuration can be found in fig. 6.4:

The waveform extracted form the ILA in fig. 6.5 illustrates the response of the pileup detection. The first pulse arrives in isolation and is filtered normally. The amplitude and rise time are captured, and the pulse state is flagged as valid. A second pulse follows once the first has fully decayed back to the baseline, and is therefore treated in the same way, producing an equally valid measurement. The third pulse, however, begins before the decay of the second has returned to the baseline. The overlap is detected, the pileup counter is incremented, and the pulse state marks the event accordingly, so that

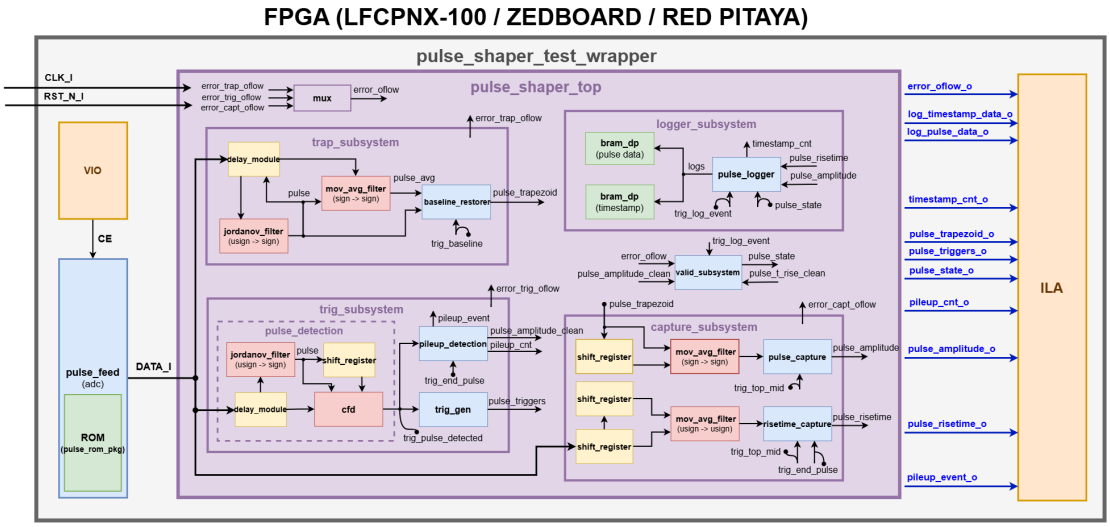


**Figure 6.4.** Block diagram of the HW testing environment for the ZedBoard.

the corrupted amplitude and rise time are not taken as valid measurements.

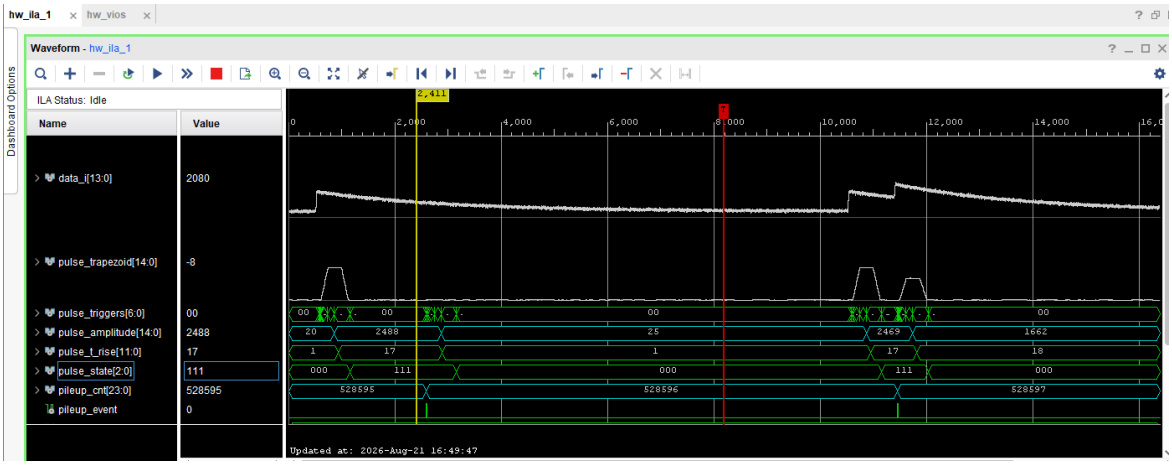


**Figure 6.5.** ILA waveform for the pulse stimulus in the ZedBoard. First two pulses are valid while the third event is affected by pileup.

## 6.3 Live Data Filtering using the Red Pitaya ADC

The last type of validation procedure is made with live acquisition of data using the ADC of the Red Pitaya Board. This project is generated through one of the scripts mentioned in section 4.1. Once generated, the Trapezoidal Pulse Shaper IP Core runs on the Red Pitaya, instantiated as a packaged IP inside the design, and the board's own ADC digitizes an analog signal produced by a signal generator replaying a specific kind of pulse shape. The results are observed with an ILA, as in the previous test cconfiguration.

The Red Pitaya IP modules can be found in the IP folder, along with other files such as its .xdc and its board. Both of these files can be find inside the constraints folder and the board folder respectively. All data related to the Red Pitaya board was obtained from Fabrice Wiotte's repository:

https://github.com/fabzz60/demo_adc_dac_Redpitaya_125_14

This is the step that validates the detection on a real and noisy signal, the behavior of the baseline restoration against a real analog baseline, and the continuous operation of the core over long periods of time. Several configurations were swept during the acquisition, varying the amplitude, the rise time

and the amount of noise, to check the limits of the Trapezoidal Pulse Shaper IP Core. A general view of the block diagram connections and the internal block design can be seen in fig. 6.6 and fig. 6.7:

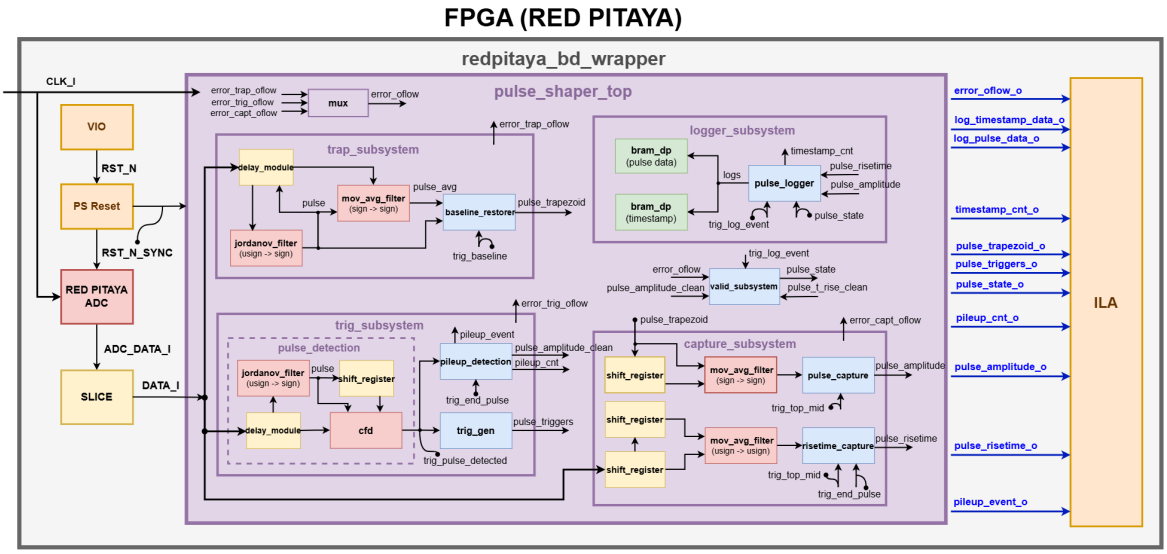


**Figure 6.6.** Red Pitaya block diagram connection with Trapezoidal Pulse Shaper IP Core.

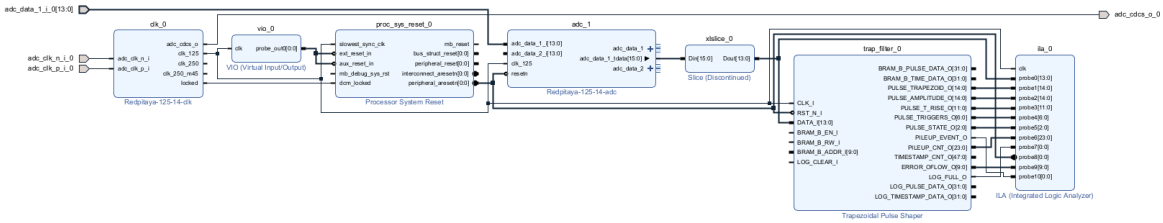

**Figure 6.7.** Red Pitaya block design view on Vivado.

> **Important.** As a reminder, the user should assert the reset signal using the VIO module every time the pulse characteristics change. Also, the user should treat the wave signal generator as unsigned to avoid any mismatch or overflow errors on the output, especially with the placement of the baseline and its noise.

In table 6.1 one can find some of the configurations input to the signal generator. The values were iterated continuously to find the limitations of the Trapezoidal Pulse Shaper IP Core. It was found that the signal generator should be used only in half of its domain, in the unsigned region to assert that the IP core is able to handle as expected unsigned input data without overflow or drift. The frequency was iterated to change the rise time of the pulses, while the levels adjusted both amplitude and baseline levels. Care should be taken to not have negative values on these parameters, especially with added noise.

**Table 6.1.** Some of the signal generator (SDG 2122X) settings used for the live data acquisition, set with noise 11.4 mV

| Signal | Frequency | High level | Low level | Phase |
|---|---|---|---|---|
| Pulse (1) | 5000 kHz | 400 mV | −300 mV | 0° |
| Pulse (2) | 5000 kHz | 980 mV | −950 mV | 0° |
| Pulse (3) | 10 000 kHz | 400 mV | −300 mV | 0° |

The general view of the physical set-up can be seen in fig. 6.8, along with some of the pulse settings tested in table 6.1.

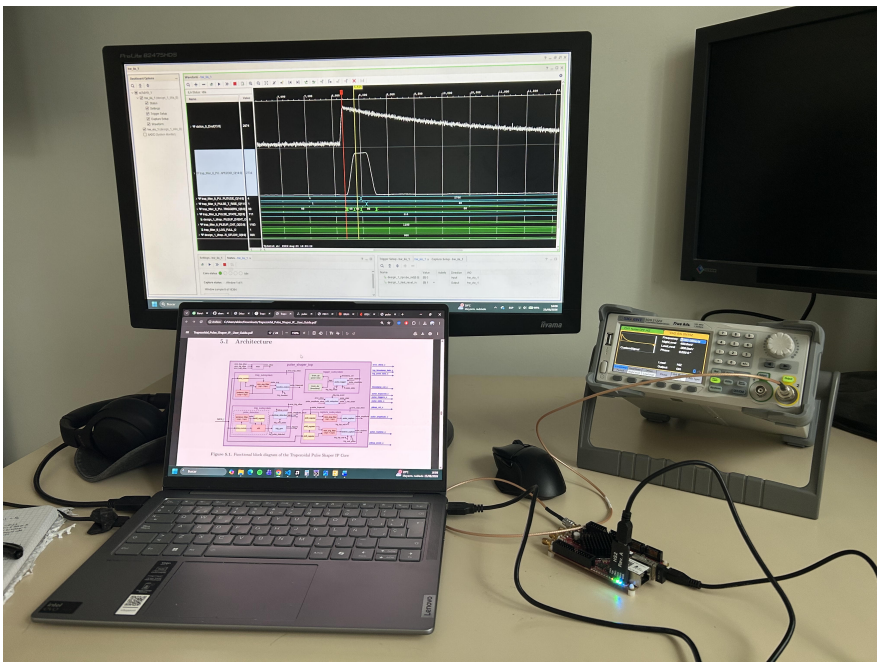

**Figure 6.8.** Red Pitaya physical set-up with Computer, Red Pitaya board, Wave Signal Generator and display of live ILA capture.

The waveform extracted from the ILA in fig. 6.9 shows, from top to bottom, the raw ADC data (unsigned) and directly below it, the corresponding trapezoidal output produced by the filter (signed). The captured amplitude and rise time counter are shown in cyan blue underneath (signed and unsigned respectively). Below these, the stage triggers mark the instants at which each stage of the trapezoid is identified, and the pulse state at the bottom confirms that the captured data has been flagged as valid.

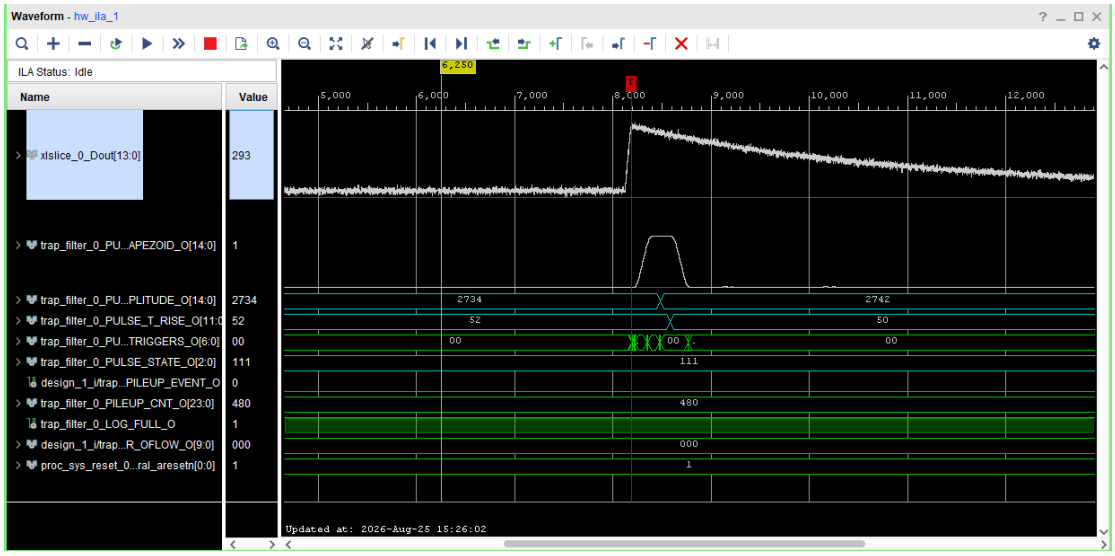


**Figure 6.9.** ILA waveform after Red Pitaya data acquisition.

## 6.4 Resource Utilization

The resources used by Trapezoidal Pulse Shaper IP Core are summarized in table 6.2. The figures were obtained after implementation with the default generics, and are dominated by the delay lines of the trapezoidal filter and the six DSP slices. Four of those DSPs are shared between the Jordanov filters (detection and slow filter), while the other two are used to compute the amplitude thresholds used in the rise time capture. The logger BRAM resource utilization will depend directly on the generics set. Both Zedboard and Red Pitaya boards carry a Zynq-7000 FPGA, and therefore share similar resource utilization reports.

**Table 6.2.** Resource utilization of Trapezoidal Pulse Shaper IP Core in the ZedBoard with the default generics

| Instance | Slice LUTs | LUT as logic | LUT as memory | Slice registers | Slices | DSPs |
|---|---|---|---|---|---|---|
| `pulse_shaper_top` | 1485 | 938 | 547 | 2685 | 792 | 6 |
| *Subsystems* | | | | | | |
| `trap_subsystem` | 619 | 357 | 262 | 1072 | 335 | 2 |
| `trig_subsystem` | 468 | 380 | 88 | 840 | 216 | 2 |
| `capture_subsystem` | 360 | 163 | 197 | 637 | 213 | 2 |
| `logger_subsystem` | 20 | 20 | 0 | 121 | 40 | 0 |

> **Note.** The remainder resources are the wiring and the output registers of the top level. The shown figures were obtained with the core implemented inside the Zedboard wrapper. The internal BRAM is also not included, as it depends directly on the implementation and the generic configuration.

The same design has been implemented on the Lattice CertusPro-NX100, whose architecture differs from the Zynq-7000. The utilization is summarized in table 6.3. As shown, the core occupies roughly 2 % of the logic and 5 % of the multipliers of the FPGA, but 96 % of the available I/O, since the full output set of `pulse_shaper_top` is brought to the pins in this configuration.

**Table 6.3.** Resource utilization of Trapezoidal Pulse Shaper IP Core on the lattice CertusPro-NX100 with the default generics

| Instance | LUT4 | Logic LUT4 | Distributed RAM | Ripple logic | PFU registers | DSP | EBR |
|---|---|---|---|---|---|---|---|
| `pulse_shaper_top` | 1570 | 420 | 210 | 940 | 1796 | 8 | 4 |
| *Subsystems* | | | | | | | |
| `trap_subsystem` | 430 | 86 | 24 | 320 | 409 | 3 | 2 |
| `trig_subsystem` | 605 | 185 | 138 | 282 | 975 | 3 | 0 |
| `capture_subsystem` | 371 | 95 | 48 | 228 | 247 | 2 | 0 |
| `logger_subsystem` | 93 | 31 | 0 | 62 | 120 | 0 | 2 |
| `valid_subsystem` | 24 | 12 | 0 | 12 | 15 | 0 | 0 |

**Table 6.4.** Timing summary after implementation on the Red Pitaya board

| Metric | Value | Comment |
|---|---|---|
| *Setup* | | |
| Worst negative slack (WNS) | 0.227 ns | Positive, constraint is met. |
| Total negative slack (TNS) | 0 ns | Zero, constraint is met. |
| Failing endpoints | 0 of 10688 | Zero, constraint is met. |
| *Hold* | | |
| Worst hold slack (WHS) | 0.042 ns | Positive, constraint is met. |
| Total hold slack (THS) | 0 ns | Zero, constraint is met. |
| Failing endpoints | 0 of 10672 | Zero, constraint is met. |
| *Pulse width* | | |
| Worst pulse width slack (WPWS) | 2.000 ns | Positive, constraint is met. |
| Total pulse width negative slack (TPWS) | 0 ns | Zero, constraint is met. |
| Failing endpoints | 0 of 5507 | Zero, constraint is met. |
| *Results* | | |
| Clock period constrained | 8 ns | 125 MHz |
| Maximum frequency | 128.7 MHz | Period minus worst setup slack. |

**Note.** All the user specified timing constraints are met. The margin on setup is small, 0.227 ns out of 8 ns. The hold margin is also positive but small, which is normal for a single clock design with no CDC.

**Table 6.5.** Power estimate after implementation on the ZedBoard Zynq-7020

| Contribution | Power | Share | Comment |
|---|---|---|---|
| Total on-chip power | 0.171 W | | |
| *Breakdown* | | | |
| Device static | 0.109 W | 63 % | Leakage, independent of the design. |
| Dynamic | 0.063 W | 37 % | Consumed by the design itself. |
| *Dynamic power by source* | | | |
| Clocks | 0.031 W | 50 % | The clock tree, the largest single contributor. |
| Signals | 0.011 W | 17 % | |
| Logic | 0.010 W | 16 % | |
| BRAM | 0.007 W | 11 % | From the debug cores, not from the IP. |
| DSP | 0.004 W | 6 % | |
| *Thermal* | | | |
| Junction temperature | 27.0 °C | | Ambient 25.0 °C. |
| Thermal margin | 58.0 °C | | Equivalent to 4.8 W of headroom. |

**Important.** The ZedBoard data come from the implemented test wrapper, which contains the ILA, the VIO and `pulse_feed` in addition to the core.

## 6.5 Known Limitations

The limitations below are ordered by how likely they are to affect a new user:

- **No run-time configuration.** Every parameter is a generic, so it is fixed when the design is synthesized. There are no configuration registers, and a change of shaping time, threshold or decay correction means running synthesis again. A system that has to follow a detector whose behavior changes during the acquisition cannot be built around this version of the core.
- **The ADC clock** and the system clock must be the same, or synchronous to each other. The core has a single clock domain and no clock crossing of any kind on the data path. If the ADC is driven by an independent oscillator, samples are lost or duplicated without any warning, and the results are wrong in a way that is hard to see. Use the clock recovered from the ADC for the whole core, or a clock derived from the same source.
- **No input data valid signal.** Every value present on `DATA_I` is treated as a new sample, so the input rate has to equal the clock rate. An ADC that delivers samples more slowly, or that pauses, has to be resampled before the core.
- **The physical parameters must match** the input pulses. The core does not check that its settings make physical sense. A flat top shorter than the charge collection time, or a decay correction that does not match the preamplifier, produces a trapezoid that looks plausible but whose amplitude is wrong. See fig. 4.3 for the shapes to compare against.
- **The first baseline rise after a reset may be logged as an event**. When the data starts arriving well after `RST_N_I` is released, the sudden step from zero to the baseline looks like a real pulse to the detection logic. It is measured and logged like any other event, with its state bits set. Discard the first logged event, or hold the reset until the input is already streaming.
- **The rise time measurement** depends on the trapezoid parameters. The capture starts once the amplitude is known, at the middle of the flat top, and it has a timeout that expires after the pulse ends. If the rise time of the raw pulse is long compared with that window, the measurement never completes and the event is marked as not valid. In practice $k$ and $m$ are always much longer than the rise time being measured, so this rarely happens, but it does mean that shortening the trapezoid also shortens the longest rise time that can be measured.
- **The overflow flags are sticky.** Once a bit of `ERROR_OFLOW_O` is raised it stays raised until `RST_N_I` is asserted again. This is useful to catch a rare saturation, but it means the flags describe the whole acquisition and not the current event, and there is no way to clear them without resetting the core.
- **The delay lines** may be inferred differently by each vendor. The shift registers use an asynchronous reset, which some synthesis tools refuse to map onto the dedicated shift register or memory resources of the device. When that happens the logic is built out of ordinary registers instead, the resource usage grows sharply and the maximum frequency drops. Check the synthesis report after the first build on a new device family.
- **The logger stops when the memory is full.** The log is a dual port BRAM without any circular buffer logic, so once the last address has been written, no further event is stored and a full flag is raised. The core keeps streaming, but every event that arrives after is lost. Recovering requires the user to assert `LOG_CLEAR_I`, which returns the write pointer to the beginning. Events could be lost between the instant the memory fills and the instant the pointer is cleared. The user should monitor the full flag, drain the memory and clear the pointer, which is extra logic or extra software that would not be needed with a circular buffer.
- **The pulse detection system relies on fixed Jordanov delays.** If the amount of delay (16 samples for k and m) is not enough to acquire the amplitude of the pulse, due to small amplitude or slow rising edge, the detection could fail. This would result on the pulse not being logged due to the noise threshold, even if properly filtered.

# A. Package Constants

These constants are defined in `trap_filter_pkg.vhd` and `they fall into two groups`: The constants declared with a range are meant to be adjusted, and any value inside that range is safe. The constants declared without a range are fixed by design. Nothing prevents them from being edited, but they are tied to each other, and changing one alone could break the timing relations that the subsystems relies on.

> **Note.** For instance, the rise time and the flat top of the fast filter, and the delays of the constant fraction discriminator, set the instant at which a pulse is declared. If they were to be changed, the delay from that instant to the baseline capture and to the start of the rising edge, `C_TRIG_DELAY_BASELINE` and `C_TRIG_DELAY_START`, need to be changed accordingly to prove that the synchronization is kept intact.

**Table A.1.** Constants of `trap_filter_pkg`

| Constant | Value | Meaning |
|---|---|---|
| *System* | | |
| `C_SYSTEM_VERSION` | 2.4 | Version of the design. |
| `C_OVERFLOW_FLAGS_DEPTH` | 9 | Highest index of `ERROR_OFLOW_O`, so 10 flags. |
| `C_TRIG_DEPTH` | 6 | Highest index of `PULSE_TRIGGERS_O`, so 7 triggers. |
| *Latency of the blocks, in clock cycles* | | |
| `C_JORDANOV_LATENCY` | 8 | Jordanov filter. |
| `C_MOVING_AVG_LATENCY` | 2 | Moving average. |
| `C_CFD_LATENCY` | 3 | Constant fraction discriminator. |
| *Detection, fast filter* | | |
| `C_FAST_JORD_K_DELAY` | 16 | Rise time of the fast trapezoid, in samples. |
| `C_FAST_JORD_M_DELAY` | 16 | Flat top of the fast trapezoid, in samples. |
| `C_TRIG_DELAY_BASELINE` | 4 | Samples from detection to the baseline capture. |
| `C_TRIG_DELAY_START` | 30 | Samples from detection to the rising edge of the slow trapezoid. |
| *Constant fraction discriminator* | | |
| `C_CFD_F_WIDTH` | 1 | Width of the scaling fraction. |
| `C_CFD_DELAY` | 32 | Delay $d$ of the algorithm, in samples. |
| `C_CFD_SLOPE_DELAY` | 8 | Delay used to check that the edge is rising. |
| `C_CFD_ZERO_TIMEOUT_WIDTH` | 7 | Timeout for the zero crossing, so 128 samples. |
| *Capture* | | |
| `C_T_RISE_WIDTH` | 12 | Width of the rise time counter. |
| `C_T_RISE_DELAY_MARGIN` | 60 | Extra delay given to the raw input for the rise time. |

| Constant | Value | Meaning |
|---|---|---|
| *Logger* | | |
| C_TIMESTAMP_CNT_WIDTH | 48 | Width of the internal time counter. |
| C_LOG_DATA_WIDTH | 32 | Width of one memory word. |
| C_LOG_MAX_AMP_WIDTH | 16 | Space reserved for the amplitude in the log word. |
| C_LOG_MAX_T_RISE_WIDTH | 12 | Space reserved for the rise time in the log word. |
| *Margin bits of the arithmetic stages* | | |
| C_SLOW_JORD_DIFF_MARGIN_BITS | 3 | Difference of the slow filter, eq. (5.1). |
| C_SLOW_JORD_ACC1_MARGIN_BITS | 2 | First accumulator of the slow filter, eq. (5.2). |
| C_SLOW_JORD_ACC2_MARGIN_BITS | 1 | Second accumulator of the slow filter, eq. (5.3). |
| C_BASE_MOV_ACC_MARGIN_BITS | 2 | Baseline moving average. |
| C_PEAK_MOV_ACC_MARGIN_BITS | 2 | Amplitude moving average. |
| C_T_RISE_MOV_ACC_MARGIN_BITS | 2 | Rise time moving average. |

**Note.** C_LOG_MAX_AMP_WIDTH is 16 bits while the amplitude output is only G_ADC_WIDTH+1 bits wide, that is 15 bits by default. The extra bit is reserved so that the format of the log word stays the same if a wider ADC is used later. To check which bits to take go to the package definition.

# B. File List

The sources are compiled in the order below into the VHDL library `trap_filter`. Some blocks are common to several subsystems (`mov_avg_filter`). The file repository can be find in the following url: https://github.com/mexarlg/trap_filter

**Table B.1.** RTL file list

| File | Description |
|---|---|
| *Package* | |
| `trap_filter_pkg.vhd` | Shared constants and helper functions. |
| `pulse_rom_pkg.vhd` | Stored virtual pulse samples. |
| *Common blocks* | |
| `shift_register.vhd` | Basic shift register. |
| `delay_module.vhd` | Delay lines for the Jordanov and moving average filters. |
| `mov_avg_filter.vhd` | Moving average filter. |
| `jordanov_filter.vhd` | Recursive Jordanov algorithm. |
| `bram_dp.vhd` | Dual port memory used by the logger. |
| *Trapezoid subsystem* | |
| `baseline_restorer.vhd` | Subtracts the baseline from the filtered data. |
| `trap_subsystem.vhd` | Groups the slow filter and the baseline restorer. |
| *Trigger subsystem* | |
| `cfd.vhd` | Constant fraction discriminator. |
| `pulse_detection.vhd` | Groups the delay line, the fast jordanov and the CFD. |
| `trig_gen.vhd` | Produces the seven triggers of table 3.3. |
| `pileup_detection.vhd` | Detects and counts the pileup events. |
| `trig_subsystem.vhd` | Groups the pulse detection, the trigger generation and the pileup detection. |
| *Capture subsystem* | |
| `pulse_capture.vhd` | Captures the amplitude on the flat top. |
| `risetime_capture.vhd` | Measures the rise time on the raw pulse. |
| `capture_subsystem.vhd` | Groups the two capture blocks. |
| *Validity and logging* | |
| `valid_subsystem.vhd` | Checks validity of events updating `PULSE_STATE_O`. |
| `pulse_logger.vhd` | Builds the log word and drives the memory. |
| `logger_subsystem.vhd` | Groups the logger, the memory and the timestamp. |
| *Top file* | |
| `pulse_shaper_top.vhd` | Top level wrapper of the IP. |
| *Testing blocks* | |
| `pulse_feed.vhd` | Stores and feeds the input samples from a pkg. |
| `pulse_shaper_test_wrapper.vhd` | Wrapper used for the Zedboard validation on hardware. |

# Bibliography

[1] Valentin T. Jordanov and Glenn F. Knoll. "Digital synthesis of pulse shapes in real time for high resolution radiation spectroscopy". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 345.2 (1994), pp. 337–345. ISSN: 0168-9002. DOI: https://doi.org/10.1016/0168-9002(94)91011-1. URL: https://www.sciencedirect.com/science/article/pii/0168900294910111.